GitOps

GitOps overview

Learn how Red Hat Advanced Cluster Management for Kubernetes integrates with Red Hat OpenShift GitOps and Argo CD so that you can manage applications across your managed clusters.

Red Hat OpenShift GitOps and Argo CD are integrated with Red Hat Advanced Cluster Management for Kubernetes. You can use the Red Hat OpenShift GitOps Operator to manage applications across your managed clusters with Argo CD.

Red Hat Advanced Cluster Management for Kubernetes supports two primary ways to use GitOps with managed clusters: the **Pull model** and the **Push model**. In the Pull model, each managed cluster runs its own Argo CD instance and syncs from Git. In the Push model, the Argo CD server on the hub cluster deploys resources directly to managed clusters.

All supported models use the `GitOpsCluster` custom resource as the primary configuration point and the `ApplicationSet` resource with a `clusterDecisionResource` generator to deploy applications across managed clusters.

Important: Red Hat Advanced Cluster Management does **not** include entitlements for OpenShift GitOps. You must have a separate OpenShift subscription for the features that you use:

- Red Hat OpenShift GitOps (excluding the Argo CD Agent) requires an **Red Hat OpenShift Container Platform** subscription.
- The Argo CD Agent requires a **Red Hat OpenShift Platform Plus** subscription. The Argo CD Agent can run on OpenShift Container Platform or on supported Kubernetes environments.

For more information, see the Self-managed Red Hat OpenShift subscription guide.

See the following topics to get started:

- Understanding the Push and Pull models
- Configuring Placement resources for GitOps
- Using the Pull model
- Using the Push model
- GitOps console
- Managing policy definitions with Red Hat OpenShift GitOps
- Deprecated APIs

Understanding the Push and Pull models

Compare the Push and Pull models for Argo CD with Red Hat Advanced Cluster Management for Kubernetes, including the advanced and `ManifestWork`-based pull variants, and confirm the subscription that each option requires.

You can use Red Hat OpenShift GitOps with Red Hat Advanced Cluster Management for Kubernetes in two primary ways: the **Pull model** and the **Push model**. Choose a model based on your network topology, security requirements, and how much status detail you need on the hub cluster.

Important: Red Hat Advanced Cluster Management does **not** include entitlements for OpenShift GitOps. You must have a separate OpenShift subscription for the features that you use:

- Red Hat OpenShift GitOps (excluding the Argo CD Agent) requires an **Red Hat OpenShift Container Platform** subscription.
- The Argo CD Agent requires a **Red Hat OpenShift Platform Plus** subscription. The Argo CD Agent can run on OpenShift Container Platform or on supported Kubernetes environments.

For more information, see the Self-managed Red Hat OpenShift subscription guide.

Model	Description	`ApplicationSet` template key	Best for	Subscription
Pull - Advanced (Red Hat OpenShift GitOps add-on with Argo CD Agent)	The add-on installs the OpenShift GitOps operator and an Argo CD instance on each managed cluster. A principal and agent architecture enables the hub Argo CD UI to show full, real-time Application status, resource trees, and logs.	`destination.name: '\{{name}\}'`	Independent cluster GitOps, network-restricted environments, enhanced security, and full Argo CD UI on the hub	Red Hat OpenShift Platform Plus (Argo CD Agent)
Pull - Basic (Red Hat OpenShift GitOps add-on)	The add-on installs the OpenShift GitOps operator and an Argo CD instance on each managed cluster. The propagation controller distributes Application resources by using `ManifestWork`. The hub shows aggregated overall health and sync status only.	`destination.server: https://kubernetes.default.svc` with pull annotations	Independent cluster GitOps, network-restricted environments, and enhanced security	Red Hat OpenShift Container Platform

Model	Description	`ApplicationSet` template key	Best for	Subscription
Push	The Argo CD server on the hub cluster deploys application resources directly on the managed clusters. Managed clusters do not run their own Argo CD instances.	`destination.server: '\{{server}\}'`	Simple setups and small fleets	Red Hat OpenShift Container Platform

After you choose a model, complete the matching procedure:

- Using the Pull model
- Using the Push model

Configuring Placement resources for GitOps

Use separate `Placement` resources for `GitOpsCluster` infrastructure and for `ApplicationSet` application delivery.

Red Hat Advanced Cluster Management uses `Placement` resources to select managed clusters for GitOps workflows. There are two distinct `Placement` resources used with GitOps:

1. **Placement for `GitOpsCluster`**: Selects which managed clusters receive the Red Hat OpenShift GitOps add-on (pull models) or are registered with the Argo CD server on the hub (push model). Referenced in the `GitOpsCluster` resource `placementRef` field.
2. **Placement for `ApplicationSet`**: Selects which managed clusters receive applications deployed by the `ApplicationSet` resource. Referenced in the `ApplicationSet` generator `labelSelector`.

These two `Placement` resources **must be separate resources** even if they select the same clusters, because they serve fundamentally different purposes. The `GitOpsCluster Placement` determines **where infrastructure is set up** (add-on deployment, cluster registration), while the `ApplicationSet Placement` determines **where applications are deployed**. You might install the add-on on all clusters but only deploy a specific application to a subset. They are also consumed by different controllers.

In some unstable network scenarios, the managed clusters might be temporarily placed in `Unavailable` state. Add tolerations to your `Placement` resources to continue to include unavailable clusters. The following example shows a `Placement` resource with tolerations:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: gitops-placement
```

```
  namespace: openshift-gitops
spec:
  tolerations:
  - key: cluster.open-cluster-management.io/unreachable
    operator: Exists
  - key: cluster.open-cluster-management.io/unavailable
    operator: Exists
  predicates:
  - requiredClusterSelector:
      labelSelector:
        matchExpressions:
        - key: vendor
          operator: "In"
          values:
          - OpenShift
        - key: local-cluster
          operator: DoesNotExist
```

Each deployment model section in this document includes the specific `Placement` resources needed. See those sections for complete end-to-end examples.

GitOps console

Learn about integrated Red Hat OpenShift GitOps console features for creating and viewing applications.

Learn more about integrated OpenShift Container Platform GitOps console features. Create and view applications, such as *ApplicationSet*, and *Argo CD* types.

- Click **Launch resource in Search** to search for related resources.
- Use *Search* to find application resources by the component `kind` for each resource.

Important: Available actions are based on your assigned role.

Querying Argo CD applications

1. Log in to your Red Hat Advanced Cluster Management hub cluster.
2. From the console header, select the *Search* icon.
3. Filter your query with the following values: `kind:application` and `apigroup:argoproj.io`.
4. Select an application to view.

Using the Push model

Use the Push model so that the Argo CD server on the hub cluster deploys application resources directly to managed clusters.

Required access: Cluster administrator

Using the Push model, the Argo CD server on the hub cluster deploys the application resources directly on the managed clusters. The managed clusters do not run their own Argo CD instances.

Note: If you plan to use the `ManifestWork`**-based pull model** or **Argo CD Agent-based pull model** with the Red Hat OpenShift GitOps add-on, you do not need to complete this section. The GitOps add-on handles all cluster registration and Argo CD deployment automatically through the `GitOpsCluster` resource with `gitopsAddon.enabled: true`.

Required access: Cluster administrator

Important: OpenShift GitOps requires an **Red Hat OpenShift Container Platform** subscription. Red Hat Advanced Cluster Management does not include this entitlement. See Understanding the Push and Pull models.

See the following topics to configure the Push model:

- Registering managed clusters to the Red Hat OpenShift GitOps operator
- Deploying an application with the Push model
- Creating a customized service account for Argo CD Push model

Registering managed clusters to the Red Hat OpenShift GitOps operator

Register managed clusters with the Argo CD server on the hub by creating a `Placement` and `GitOpsCluster` resource for the Push model.

Required access: Cluster administrator

Note: Cluster registration as described in this section is for the Push model only. The Pull model handles registration automatically through the `GitOpsCluster` resource.

To configure OpenShift GitOps with the Push model, register a set of one or more Red Hat Advanced Cluster Management for Kubernetes managed clusters to an instance of the OpenShift GitOps operator.

Prerequisites

- Install the Red Hat OpenShift GitOps operator on your Red Hat Advanced Cluster Management hub cluster.
- Import one or more managed clusters.
- Create a `ManagedClusterSet` and bind it to the namespace where OpenShift GitOps is deployed. See Creating a *ManagedClusterSet* and Creating a *ManagedClusterSetBinding* resource.
- Enable the `ManagedServiceAccount` add-on to rotate the token that the Argo CD Push model uses to connect to the managed cluster.

Procedure

Complete the following steps to register managed clusters to OpenShift GitOps:

1. Create a `Placement` resource for the `GitOpsCluster` to select which managed clusters to register. Add the following YAML sample:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: gitops-placement
  namespace: openshift-gitops
spec:
  tolerations:
  - key: cluster.open-cluster-management.io/unreachable
    operator: Exists
  - key: cluster.open-cluster-management.io/unavailable
    operator: Exists
  predicates:
  - requiredClusterSelector:
      labelSelector:
        matchExpressions:
        - key: vendor
          operator: "In"
          values:
          - OpenShift
        - key: local-cluster
          operator: DoesNotExist
```

2. Create a `GitOpsCluster` custom resource to register the managed clusters by adding the following YAML sample:

```
apiVersion: apps.open-cluster-management.io/v1beta1
kind: GitOpsCluster
metadata:
  name: gitops-cluster-push
  namespace: openshift-gitops
spec:
  argoServer:
    argoNamespace: openshift-gitops
  placementRef:
    kind: Placement
    apiVersion: cluster.open-cluster-management.io/v1beta1
    name: gitops-placement
```

3. Apply the YAML samples and save your changes.

The `GitOpsCluster` controller automatically creates the following resources during reconciliation:

- The `acm-placement` duck-type `ConfigMap` in the `argoServer.argoNamespace` - this is the ConfigMap referenced by `configMapRef` in the `ApplicationSet clusterDecisionResource` generator. It tells the `ApplicationSet` controller how to read `PlacementDecision` resources. The auto-created ConfigMap

looks like the following:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: acm-placement
  namespace: openshift-gitops
data:
  apiVersion: cluster.open-cluster-management.io/v1beta1
  kind: placementdecisions
  statusListKey: decisions
  matchKey: clusterName
```

- A `Role` and `RoleBinding` granting the `ApplicationSet` controller service account `list` access to `PlacementDecision` and `PlacementRule` resources in the Argo CD namespace. This is required for the `clusterDecisionResource` generator to read the `PlacementDecision` resources that the `Placement` controller creates.

These resources are automatically created - you do not need to create them manually. If you need to verify that they were created, run the following commands:

```
oc get configmap acm-placement -n openshift-gitops -o yaml
oc get role openshift-gitops-applicationset-controller-placement -n openshift-gitops
oc get rolebinding openshift-gitops-applicationset-controller-placement -n openshift-
gitops
```

Registering non-OpenShift Container Platform clusters to Red Hat OpenShift GitOps

Note: This capability is for the Push model only.

You can use the `GitOpsCluster` resource to register a non-OpenShift Container Platform cluster. If the API server URL in the `ManagedCluster` resource `spec` is empty, update it manually by running the following command:

```
oc edit managedclusters <cluster-name>
```

1. Verify that the `spec` includes the client configuration:

```
spec:
  managedClusterClientConfigs:
  - caBundle: <base64-encoded-ca>
    url: https://<api-server-url>
```

Red Hat OpenShift GitOps token

Note: The token and cluster secret mechanisms described in this section are for the Push model

only.

When you integrate with the OpenShift GitOps operator, for every managed cluster bound through the placement and `ManagedClusterSetBinding`, a secret with a token to access the `ManagedCluster` is created in the OpenShift GitOps namespace. The secure cluster secrets are rendered based on the following priority order:

Cluster proxy service: The default option because the `cluster-proxy` add-on is always enabled in multicluster engine operator. Secrets include the cluster proxy URL, service CA, and `ManagedServiceAccount application-manager` token.

Managed cluster client configurations: If the cluster proxy is unavailable. Secrets include the API server URL and CA from the managed cluster specification, plus the same `ManagedServiceAccount` token.

If you do not want the default service account, see *Creating a customized service account for Argo CD push model*.

Deploying an application with the Push model

Deploy applications from the hub Argo CD server to managed clusters by using an `ApplicationSet` resource.

Required access: Cluster administrator

In the Push model, the hub Argo CD pushes resources directly to managed clusters. The `ApplicationSet` uses `destination.server: '\{{server}\}'` to target each managed cluster's API server.

Prerequisites

- Complete the *Registering managed clusters to Red Hat OpenShift GitOps operator* procedure.
- If your `openshift-gitops-argocd-application-controller` service account is *not* assigned as a cluster administrator, add `ClusterRoleBinding` privileges. The `cluster-admin ClusterRole` is only an example - see the Red Hat OpenShift GitOps documentation for configuring least-privilege permissions.

Procedure

Complete the following steps:

1. Create a separate `Placement` resource for the `ApplicationSet` to select which managed clusters receive the application. This `Placement` is distinct from the `GitOpsCluster Placement`. See Configuring Placement resources for GitOps for details. Add the following YAML sample:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: app-placement
```

```
  namespace: openshift-gitops
spec:
  tolerations:
  - key: cluster.open-cluster-management.io/unreachable
    operator: Exists
  - key: cluster.open-cluster-management.io/unavailable
    operator: Exists
  predicates:
  - requiredClusterSelector:
      labelSelector:
        matchExpressions:
        - key: vendor
          operator: "In"
          values:
          - OpenShift
        - key: local-cluster
          operator: DoesNotExist
```

2. Apply the YAML sample by running the following command:

```
oc apply -f app-placement.yaml
```

3. Create the `ApplicationSet` resource. In the Push model, use `destination.server: '\{{server}\}'` to push application resources directly from the hub to each managed cluster. See the following example:

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: guestbook-allclusters
  namespace: openshift-gitops
spec:
  generators:
  - clusterDecisionResource:
      configMapRef: acm-placement
      labelSelector:
        matchLabels:
          cluster.open-cluster-management.io/placement: app-placement
      requeueAfterSeconds: 30
  template:
    metadata:
      name: '{{name}}-guestbook'
    spec:
      destination:
        namespace: guestbook
        server: '{{server}}'
      project: default
      source:
```

```
        repoURL: https://github.com/argoproj/argocd-example-apps.git
        targetRevision: HEAD
        path: guestbook
      syncPolicy:
        automated:
          prune: true
          selfHeal: true
        syncOptions:
        - CreateNamespace=true
```

Note: The `destination.server: '\{{server}\}'` field is what distinguishes the Push model from the pull models. The hub Argo CD directly communicates with each managed cluster's API server. The `configMapRef: acm-placement` references the duck-type `ConfigMap` that the `GitOpsCluster` controller creates automatically during reconciliation.

1. Apply the YAML sample by running the following command:

```
oc apply -f applicationset.yaml
```

2. Verify the application is deployed by running the following command:

```
oc get applications.argoproj.io -n openshift-gitops
```

Important: With a large number of managed clusters, consider potential strain on the OpenShift GitOps controller memory and CPU usage. To optimize resource management, see Configuring resource quota or requests in the Red Hat OpenShift GitOps documentation.

Creating a customized service account for Argo CD Push model

Create a customized service account when you need a limited set of permissions for the Argo CD Push model on managed clusters.

Required access: Cluster administrator

Note: This capability is for the Push model only.

Create a service account on a managed cluster by creating a `ManagedServiceAccount` resource on the hub cluster. By using a different service account, you can control the permissions granted instead of using the default `application-manager` service account.

Update the `GitOpsCluster` resource to use the managed service account by adding the `managedServiceAccountRef` property:

```
apiVersion: apps.open-cluster-management.io/v1beta1
kind: GitOpsCluster
```

```
metadata:
  name: argo-acm-importer
  namespace: openshift-gitops
spec:
  managedServiceAccountRef: <managed-sa-sample>
  argoServer:
    argoNamespace: openshift-gitops
  placementRef:
    kind: Placement
    apiVersion: cluster.open-cluster-management.io/v1beta1
    name: gitops-placement
    namespace: openshift-gitops
```

Verify that the new Argo CD cluster secret is created by running the following command:

```
oc get secrets -n openshift-gitops <managed-cluster>-<managed-sa-sample>-cluster-
secret
```

Using the Pull model

Use the Pull model so that each managed cluster runs Argo CD locally and syncs applications from Git repositories.

Required access: Cluster administrator

In the Pull model, the Red Hat OpenShift GitOps add-on installs the OpenShift GitOps operator and an Argo CD instance on each managed cluster. Each instance syncs from Git repositories independently. Red Hat Advanced Cluster Management for Kubernetes supports two Pull variants:

- **Argo CD Agent-based pull** (Argo CD Agent): Full, real-time Application status on the hub through a principal and agent architecture. Requires a Red Hat OpenShift Platform Plus subscription.
- `ManifestWork`**-based pull**: Aggregated overall health and sync status on the hub through `ManifestWork`. Requires an Red Hat OpenShift Container Platform subscription.

Important: Red Hat Advanced Cluster Management does **not** include entitlements for OpenShift GitOps. You must have a separate OpenShift subscription for the features that you use:

- Red Hat OpenShift GitOps (excluding the Argo CD Agent) requires an **Red Hat OpenShift Container Platform** subscription.
- The Argo CD Agent requires a **Red Hat OpenShift Platform Plus** subscription. The Argo CD Agent can run on OpenShift Container Platform or on supported Kubernetes environments.

For more information, see the Self-managed Red Hat OpenShift subscription guide.

Important: If you enable the OpenShift GitOps add-on by using the `GitOpsCluster` custom resource, then the `GitOpsCluster` disables the Push model for all applications.

See the following topics to configure the Pull model:

- Deploying Argo CD with the Argo CD Agent-based pull model (Argo CD Agent)
- Deploying Argo CD with the `ManifestWork`-based pull model
- Managing the Red Hat OpenShift GitOps add-on
- Deploying the Argo CD *ApplicationSet* resource in any namespace (Technology Preview)
- Implementing progressive rollout strategy by using *ApplicationSet* resource (Technology Preview)

Deploying Argo CD with the Argo CD Agent-based pull model (Argo CD Agent)

Use the Argo CD Agent-based pull model so that each managed cluster runs Argo CD locally while the hub Argo CD UI shows full, real-time application status through the Argo CD Agent.

Required access: Cluster administrator

Technology Preview: The Argo CD Agent mode is a Technology Preview feature.

Important: The Argo CD Agent requires a **Red Hat OpenShift Platform Plus** subscription. Red Hat Advanced Cluster Management does not include this entitlement. See Understanding the Push and Pull models.

Use the Argo CD Agent-based pull model to enable the Red Hat OpenShift GitOps add-on with the Argo CD Agent. Like the `ManifestWork`-based pull model, this mode installs the OpenShift GitOps operator and an independent Argo CD instance on each managed cluster. Additionally, the principal and agent architecture enables the hub Argo CD UI to show full, real-time Application status, resource trees, and logs from all managed clusters - providing the same Argo CD UI experience as the Push model, while keeping the security and network benefits of the pull architecture.

The Argo CD Agent-based pull model uses a **principal and agent** architecture:

- **Principal** (hub): Runs in the `openshift-gitops` namespace on the hub. It receives connections from agents, dispatches applications, and aggregates status.
- **Agent** (managed cluster): Runs on each managed cluster. It connects to the principal, receives application specs (managed mode) or syncs local applications back to the hub (autonomous mode).
- **Application controller** (managed cluster): Reconciles applications locally on each managed cluster, regardless of mode.
- The principal uses destination-based mapping to route applications to the correct agent based on `destination.name`.

Managed versus autonomous mode

Aspect	Managed mode	Autonomous mode
Source of truth	Hub (principal)	Spoke (workload cluster)
Application creation	On the hub; principal dispatches to agents	On the managed cluster (by using Policy or Git)
Application reconciliation	Local on spoke	Local on spoke; agent syncs specs and status to hub
Hub capability	Full control. Create, modify, and delete applications	Read-only mirror. Inspect only

See the following topics to configure the Argo CD Agent-based pull model:

- Configuring the hub cluster for Argo CD Agent
- Enabling the Red Hat OpenShift GitOps add-on with Argo CD Agent in managed mode
- Enabling the Red Hat OpenShift GitOps add-on with Argo CD Agent in autonomous mode
- Configuring agent version drift heal

Configuring the hub cluster for Argo CD Agent

Configure the OpenShift GitOps operator subscription and the hub `ArgoCD` custom resource before you create the `GitOpsCluster` resource for the Argo CD Agent.

Required access: Cluster administrator

Prerequisites

- Red Hat Advanced Cluster Management hub cluster installed.
- Managed clusters registered with Red Hat Advanced Cluster Management.
- Red Hat OpenShift GitOps operator installed on the hub cluster.
- `ManagedClusterSet` bound to the target namespace.

Before creating the `GitOpsCluster` resource, configure the Red Hat OpenShift GitOps operator subscription and the `ArgoCD` custom resource on the hub cluster. Complete the following steps:

Procedure

1. On the hub cluster, modify the Red Hat OpenShift GitOps operator subscription to include the required environment variables by running the following command:

```
oc edit subscription openshift-gitops-operator -n openshift-gitops-operator
```

2. Add the following environment variables to the `spec.config.env` field by adding the following YAML sample:

```
spec:
  config:
```

```
    env:
    - name: ARGOCD_CLUSTER_CONFIG_NAMESPACES
      value: "openshift-gitops"
    - name: ARGOCD_PRINCIPAL_TLS_SERVER_ALLOW_GENERATE
      value: "false"
    - name: ARGOCD_PRINCIPAL_REDIS_SERVER_ADDRESS
      value: openshift-gitops-redis:6379
```

3. Replace the existing `ArgoCD` custom resource on the hub cluster with the compatible Agent mode configuration by adding the following YAML sample:

```
apiVersion: argoproj.io/v1beta1
kind: ArgoCD
metadata:
  name: openshift-gitops
  namespace: openshift-gitops
spec:
  controller:
    enabled: false
  sourceNamespaces:
  - '*'
  argoCDAgent:
    principal:
      enabled: true
      auth: "mtls:CN=system:open-cluster-management:cluster:([^:]+):addon:gitops-
addon:agent:gitops-addon-agent"
      namespace:
        allowedNamespaces:
        - '*'
      server:
        route:
          enabled: true
```

4. Apply the YAML sample by running the following command:

```
oc apply -f argocd-hub.yaml
```

Note: This configuration disables the traditional `ArgoCD` controller and enables the agent principal with mutual TLS authentication. The `sourceNamespaces` field with `*` allows applications from any namespace. The `server.route.enabled` field creates an OpenShift Container Platform Route for the agent principal, which is used for agent-to-hub communication.

1. Configure the `default AppProject` in `openshift-gitops` with wildcard settings. The principal only propagates `AppProject` resources whose destinations and `sourceNamespaces` match the agent's configuration. Without wildcards, propagation is skipped and agents cannot find the `AppProject` for their applications. Apply the following YAML sample:

```
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: default
  namespace: openshift-gitops
spec:
  sourceNamespaces:
  - '*'
  destinations:
  - name: '*'
    namespace: '*'
    server: '*'
  clusterResourceWhitelist:
  - group: '*'
    kind: '*'
  sourceRepos:
  - '*'
```

2. Apply the YAML sample by running the following command:

```
oc apply -f appproject.yaml
```

3. Verify that a `ManagedClusterSetBinding` for the `ManagedClusterSet` exists in the `openshift-gitops` namespace. Without it, the `Placement` controller cannot find any managed clusters. See the following YAML sample:

```
apiVersion: cluster.open-cluster-management.io/v1beta2
kind: ManagedClusterSetBinding
metadata:
  name: default
  namespace: openshift-gitops
spec:
  clusterSet: default
```

Enabling the Red Hat OpenShift GitOps add-on with Argo CD Agent in managed mode

In managed mode, the hub is the source of truth. Applications are created on the hub and the principal dispatches them to agents for local reconciliation on managed clusters.

Required access: Cluster administrator

In managed mode, the hub is the source of truth. Applications are created on the hub and the principal dispatches them to agents for local reconciliation on managed clusters.

Procedure

Complete the following steps:

1. Create a `Placement` resource for the `GitOpsCluster` to select which managed clusters receive the add-on by adding the following YAML sample:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: gitops-placement
  namespace: openshift-gitops
spec:
  tolerations:
  - key: cluster.open-cluster-management.io/unreachable
    operator: Exists
  - key: cluster.open-cluster-management.io/unavailable
    operator: Exists
  predicates:
  - requiredClusterSelector:
      labelSelector:
        matchExpressions:
        - key: vendor
          operator: "In"
          values:
          - OpenShift
        - key: local-cluster
          operator: DoesNotExist
```

2. Apply the YAML sample by running the following command:

```
oc apply -f gitops-placement.yaml
```

3. Create a `GitOpsCluster` resource with the Argo CD Agent enabled by adding the following YAML sample:

```
apiVersion: apps.open-cluster-management.io/v1beta1
kind: GitOpsCluster
metadata:
  name: agent-gitops
  namespace: openshift-gitops
spec:
  argoServer:
    argoNamespace: openshift-gitops
  placementRef:
    kind: Placement
    apiVersion: cluster.open-cluster-management.io/v1beta1
    name: gitops-placement
  gitopsAddon:
    enabled: true
```

```
    argoCDAgent:
      enabled: true
```

Note: The `mode` field determines the agent operation mode. The default value is `managed`. For OpenShift Container Platform managed clusters, you can enable OLM-based deployment by adding `olmSubscription.enabled: true` to the `gitopsAddon` specification in case the add-on auto-detection of the cluster type is not working properly. For more information, see *Managing the Red Hat OpenShift GitOps add-on*.

4. Apply the YAML sample by running the following command:

```
oc apply -f gitopscluster.yaml
```

The `GitOpsCluster` controller automatically creates the following resources during reconciliation: The `acm-placement` duck-type `ConfigMap` in the `argoServer.argoNamespace` - this is the ConfigMap referenced by `configMapRef` in the `ApplicationSet clusterDecisionResource` generator. It tells the `ApplicationSet` controller how to read `PlacementDecision` resources. The auto-created ConfigMap looks like the following:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: acm-placement
  namespace: openshift-gitops
data:
  apiVersion: cluster.open-cluster-management.io/v1beta1
  kind: placementdecisions
  statusListKey: decisions
  matchKey: clusterName
```

   - A `Role` and `RoleBinding` granting the `ApplicationSet` controller service account `list` access to `PlacementDecision` and `PlacementRule` resources in the Argo CD namespace. This is required for the `clusterDecisionResource` generator to read the `PlacementDecision` resources that the `Placement` controller creates.

5. These resources are automatically created - you do not need to create them manually. If you need to verify that they were created, run the following commands:

```
oc get configmap acm-placement -n openshift-gitops -o yaml
oc get role openshift-gitops-applicationset-controller-placement -n openshift-
gitops
oc get rolebinding openshift-gitops-applicationset-controller-placement -n
openshift-gitops
```

6. Customize the auto-generated `ArgoCD Policy` to add RBAC for the managed clusters. The controller creates a `Policy` named `<gitopscluster-name>-argocd-policy` that configures the `ArgoCD`

custom resource on managed clusters. This `Policy` is for spoke cluster configuration only. Export the `Policy`, add a `ClusterRoleBinding`, and re-apply it. The following example shows the complete `Policy` with RBAC added as a second policy-template:

```
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: agent-gitops-argocd-policy
  namespace: openshift-gitops
  annotations:
    policy.open-cluster-management.io/standards: NIST-CSF
    policy.open-cluster-management.io/categories: PR.PT Protective Technology
    policy.open-cluster-management.io/controls: PR.PT-3 Least Functionality
spec:
  remediationAction: enforce
  disabled: false
  policy-templates:
    - objectDefinition:
        apiVersion: policy.open-cluster-management.io/v1
        kind: ConfigurationPolicy
        metadata:
          name: agent-gitops-argocd-config-policy
        spec:
          pruneObjectBehavior: None
          remediationAction: enforce
          severity: medium
          object-templates:
            - complianceType: musthave
              objectDefinition:
                apiVersion: argoproj.io/v1beta1
                kind: ArgoCD
                metadata:
                  name: acm-openshift-gitops
                  namespace: openshift-gitops
                  labels:
                    apps.open-cluster-management.io/gitopsaddon: "true"
                spec:
                  server:
                    enabled: false
                  argoCDAgent:
                    agent:
                      enabled: true
                      allowedNamespaces:
                        - "*"
                      client:
                        principalServerAddress: "<auto-discovered>"
                        principalServerPort: "443"
                        mode: "managed"
                      tls:
                        secretName: argocd-agent-client-tls
```

```
                        rootCASecretName: argocd-agent-ca
    - objectDefinition:
        apiVersion: policy.open-cluster-management.io/v1
        kind: ConfigurationPolicy
        metadata:
          name: agent-gitops-argocd-rbac-policy
        spec:
          pruneObjectBehavior: None
          remediationAction: enforce
          severity: medium
          object-templates:
            - complianceType: musthave
              objectDefinition:
                apiVersion: rbac.authorization.k8s.io/v1
                kind: ClusterRoleBinding
                metadata:
                  name: acm-openshift-gitops-cluster-admin
                roleRef:
                  apiGroup: rbac.authorization.k8s.io
                  kind: ClusterRole
                  name: cluster-admin
                subjects:
                  - kind: ServiceAccount
                    name: acm-openshift-gitops-argocd-application-controller
                    namespace: openshift-gitops
```

7. Apply the YAML sample by running the following command:

```
oc apply -f policy.yaml
```

Important: The `cluster-admin ClusterRole` in this example grants full cluster administrator privileges. This is only an example. For production environments, see the Red Hat OpenShift GitOps documentation for configuring the Argo CD application controller with appropriate least-privilege permissions for managing resources outside its namespace. The `principalServerAddress` is auto-populated by the controller - export the auto-generated `Policy` with `oc get policy agent-gitops-argocd-policy -n openshift-gitops -o yaml` to see the actual value.

8. Create a separate `Placement` resource for the `ApplicationSet` to select which managed clusters receive the application. This `Placement` is distinct from the `GitOpsCluster Placement` created in step 1. See Configuring Placement resources for GitOps for details. Add the following YAML sample:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: app-placement
  namespace: openshift-gitops
```

```
spec:
  tolerations:
  - key: cluster.open-cluster-management.io/unreachable
    operator: Exists
  - key: cluster.open-cluster-management.io/unavailable
    operator: Exists
  predicates:
  - requiredClusterSelector:
      labelSelector:
        matchExpressions:
        - key: vendor
          operator: "In"
          values:
          - OpenShift
        - key: local-cluster
          operator: DoesNotExist
```

9. Apply the YAML sample by running the following command:

```
oc apply -f app-placement.yaml
```

10. Deploy applications by using an `ApplicationSet` resource on the hub cluster. In the Argo CD Agent-based pull model, use `destination.name` so the principal routes the application to the correct agent. See the following YAML sample:

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: guestbook-appset
  namespace: openshift-gitops
spec:
  generators:
  - clusterDecisionResource:
      configMapRef: acm-placement
      labelSelector:
        matchLabels:
          cluster.open-cluster-management.io/placement: app-placement
      requeueAfterSeconds: 30
  template:
    metadata:
      name: '{{name}}-guestbook'
    spec:
      project: default
      source:
        repoURL: https://github.com/argoproj/argocd-example-apps
        targetRevision: HEAD
        path: guestbook
      destination:
```

```
        name: '{{name}}'
        namespace: guestbook
      syncPolicy:
        automated:
          prune: true
          selfHeal: true
        syncOptions:
        - CreateNamespace=true
```

Note: The `destination.name: '\{{name}\}'` field is what distinguishes the Argo CD Agent-based pull model from the other models. The principal uses destination-based mapping to route applications to the correct agent. The `configMapRef: acm-placement` references the duck-type `ConfigMap` that the `GitOpsCluster` controller creates automatically during reconciliation.

1. Apply the YAML sample by running the following command:

```
oc apply -f applicationset.yaml
```

2. Verify the deployment by completing the following steps:

   a. Verify the agent principal is running on the hub cluster by running the following command:

```
oc get pods -n openshift-gitops -l app.kubernetes.io/name=openshift-gitops-
agent-principal
```

   b. On the managed cluster, verify the agent is running by running the following command:

```
oc get pods -n openshift-gitops -l app.kubernetes.io/part-of=argocd-agent
--context <managed-cluster-context>
```

   c. On the hub cluster, verify that the application status shows `Synced` by running the following command:

```
oc get applications.argoproj.io -n openshift-gitops
```

Enabling the Red Hat OpenShift GitOps add-on with Argo CD Agent in autonomous mode

In autonomous mode, the managed cluster is the source of truth. Applications are created on the managed cluster, and the agent syncs specs and status back to the hub principal as a read-only mirror.

Required access: Cluster administrator

In autonomous mode, the spoke cluster is the source of truth. Applications are created on the

managed cluster, and the agent syncs specs and status back to the hub principal, which acts as a read-only mirror. This mode is recommended for App of Apps workflows where all ongoing changes flow through Git after initial bootstrap.

Procedure

Complete the following steps:

1. Create a `GitOpsCluster` resource with autonomous mode by adding the following YAML sample:

```
apiVersion: apps.open-cluster-management.io/v1beta1
kind: GitOpsCluster
metadata:
  name: autonomous-gitops
  namespace: openshift-gitops
spec:
  argoServer:
    argoNamespace: openshift-gitops
  placementRef:
    kind: Placement
    apiVersion: cluster.open-cluster-management.io/v1beta1
    name: gitops-placement
  gitopsAddon:
    enabled: true
    argoCDAgent:
      enabled: true
      mode: autonomous
```

2. Apply the YAML sample by running the following command:

```
oc apply -f gitopscluster.yaml
```

3. Customize the auto-generated `ArgoCD Policy` to add RBAC for the managed clusters, following the same pattern as the managed mode example. Export the auto-generated `Policy`, add a `ClusterRoleBinding` as a second policy-template, and re-apply it.

Important: Do not add `Application` resources to the auto-generated `ArgoCD Policy`. The auto-generated `Policy` is for configuring the spoke cluster (`ArgoCD` settings, RBAC) only. For deploying applications, use one of the following approaches.

1. Deploy applications to the managed cluster by using one of the following methods:
   - **Create a separate OCM Policy** to deploy an App of Apps bootstrap `Application` to managed clusters. This is a new `Policy`, separate from the auto-generated `ArgoCD Policy`. After the bootstrap `Application` is deployed, all ongoing changes flow through Git. See the following YAML sample for the bootstrap policy:

```
apiVersion: policy.open-cluster-management.io/v1
```

```
kind: Policy
metadata:
  name: autonomous-app-bootstrap
  namespace: openshift-gitops
spec:
  remediationAction: enforce
  disabled: false
  policy-templates:
    - objectDefinition:
        apiVersion: policy.open-cluster-management.io/v1
        kind: ConfigurationPolicy
        metadata:
          name: autonomous-app-bootstrap-config
        spec:
          pruneObjectBehavior: None
          remediationAction: enforce
          severity: medium
          object-templates:
            - complianceType: musthave
              objectDefinition:
                apiVersion: argoproj.io/v1alpha1
                kind: Application
                metadata:
                  name: app-of-apps-root
                  namespace: openshift-gitops
                spec:
                  project: default
                  source:
                    repoURL: https://github.com/YOUR-ORG/YOUR-GITOPS-REPO
                    targetRevision: HEAD
                    path: apps
                  destination:
                    server: https://kubernetes.default.svc
                    namespace: openshift-gitops
                  syncPolicy:
                    automated:
                      prune: true
                      selfHeal: true
```

- **Use Git** to manage applications after initial bootstrap. With autonomous mode, the managed cluster Argo CD watches Git repositories and syncs changes automatically. The agent syncs application specs and status back to the hub principal for visibility.

  1. On the hub cluster, verify that autonomous applications are visible as read-only mirrors by running the following command:

```
oc get applications.argoproj.io -n openshift-gitops
```

Configuring agent version drift heal

Configure how the add-on responds when the Argo CD Agent version on a managed cluster drifts from the expected version.

Required access: Cluster administrator

In agent mode, the hub controller watches the `ArgoCD` principal `Deployment` for container image changes. When the Red Hat OpenShift GitOps Operator upgrades the principal (changing the image), the controller automatically patches the `ArgoCD Policy` to enforce the correct `spec.argoCDAgent.agent.image` on managed clusters. This ensures spoke agents stay version-compatible with the hub principal without manual intervention.

- Disable drift heal by setting the `apps.open-cluster-management.io/skip-agent-version-heal: "true"` annotation on the `GitOpsCluster`.

Additional resources

- Continue setting up the Red Hat OpenShift GitOps add-on by completing *Managing the Red Hat OpenShift GitOps add-on*.

Deploying Argo CD with the `ManifestWork`-based pull model

Use the `ManifestWork`-based pull model to install the OpenShift GitOps operator and an independent Argo CD instance on each managed cluster, with application status aggregated on the hub through `ManifestWork`.

Required access: Cluster administrator

The `ManifestWork`-based pull model uses the Red Hat OpenShift GitOps add-on to install the OpenShift GitOps operator and an Argo CD instance on each managed cluster. Each instance syncs from Git repositories independently. The propagation controller on the hub distributes `Application` resources to managed clusters via `ManifestWork`. The hub shows aggregated overall health and sync status for each application.

Note: The `ManifestWork`-based pull model provides overall Application health and sync status on the hub. If you need full Argo CD UI compatibility with real-time resource trees, logs, and detailed status from managed clusters, use the *Argo CD Agent-based pull model* with the Argo CD Agent instead.

To enable the `ManifestWork`-based pull model, create a `GitOpsCluster` resource with `gitopsAddon.enabled: true`. The controller creates an `ArgoCD Policy` named `<gitopscluster-name>-argocd-policy` that deploys and configures the `ArgoCD` custom resource on managed clusters through the Red Hat Advanced Cluster Management governance framework. The `Policy` is user-owned after creation - you can modify it for RBAC or `ArgoCD` settings. The controller does not overwrite your customizations.

Required access: Cluster administrator

Important: The `ManifestWork`-based pull model is only supported on OpenShift Container Platform managed clusters. For managed clusters that are not OpenShift Container Platform, use the Argo CD Agent-based pull model with the Argo CD Agent.

Important: OpenShift GitOps requires an **Red Hat OpenShift Container Platform** subscription. Red Hat Advanced Cluster Management does not include this entitlement. See Understanding the Push and Pull models.

See the following topics:

- Enabling the Red Hat OpenShift GitOps add-on for the `ManifestWork`-based pull model
- Customizing the Argo CD policy for the `ManifestWork`-based pull model
- Deploying an application with the `ManifestWork`-based pull model

Enabling the Red Hat OpenShift GitOps add-on for the `ManifestWork`-based pull model

Create the `Placement` and `GitOpsCluster` resources so that the Red Hat OpenShift GitOps add-on deploys Argo CD on the selected managed clusters.

Required access: Cluster administrator

Prerequisites

- Red Hat Advanced Cluster Management hub cluster installed.
- Managed clusters registered with Red Hat Advanced Cluster Management.
- Red Hat OpenShift GitOps operator installed on the hub cluster.
- `ManagedClusterSet` bound to the target namespace.

Procedure

1. Create a `ManagedClusterSetBinding` resource to bind the `ManagedClusterSet` to the `openshift-gitops` namespace by adding the following YAML sample:

```
apiVersion: cluster.open-cluster-management.io/v1beta2
kind: ManagedClusterSetBinding
metadata:
  name: default
  namespace: openshift-gitops
spec:
  clusterSet: default
```

2. Apply the YAML sample by running the following command:

```
oc apply -f managedclustersetbinding.yaml
```

3. Create a `Placement` resource for the `GitOpsCluster` to select which managed clusters receive the

add-on by adding the following YAML sample:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: gitops-placement
  namespace: openshift-gitops
spec:
  tolerations:
  - key: cluster.open-cluster-management.io/unreachable
    operator: Exists
  - key: cluster.open-cluster-management.io/unavailable
    operator: Exists
  predicates:
  - requiredClusterSelector:
      labelSelector:
        matchExpressions:
        - key: vendor
          operator: "In"
          values:
          - OpenShift
        - key: local-cluster
          operator: DoesNotExist
```

4. Apply the YAML sample by running the following command:

```
oc apply -f gitops-placement.yaml
```

5. Create a `GitOpsCluster` resource to enable the Red Hat OpenShift GitOps add-on by adding the following YAML sample:

```
apiVersion: apps.open-cluster-management.io/v1beta1
kind: GitOpsCluster
metadata:
  name: gitops-clusters
  namespace: openshift-gitops
spec:
  argoServer:
    argoNamespace: openshift-gitops
  placementRef:
    kind: Placement
    apiVersion: cluster.open-cluster-management.io/v1beta1
    name: gitops-placement
  gitopsAddon:
    enabled: true
```

6. Apply the YAML sample by running the following command:

```
oc apply -f gitopscluster.yaml
```

The GitOpsCluster controller automatically creates the following resources during reconciliation:

- The acm-placement duck-type ConfigMap in the argoServer.argoNamespace - this is the ConfigMap referenced by configMapRef in the ApplicationSet clusterDecisionResource generator. It tells the ApplicationSet controller how to read PlacementDecision resources. The auto-created ConfigMap looks like the following:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: acm-placement
  namespace: openshift-gitops
data:
  apiVersion: cluster.open-cluster-management.io/v1beta1
  kind: placementdecisions
  statusListKey: decisions
  matchKey: clusterName
```

- A Role and RoleBinding granting the ApplicationSet controller service account list access to PlacementDecision and PlacementRule resources in the Argo CD namespace. This is required for the clusterDecisionResource generator to read the PlacementDecision resources that the Placement controller creates.
- An ArgoCD Policy named <gitopscluster-name>-argocd-policy that configures the ArgoCD custom resource on managed clusters.

These resources are automatically created - you do not need to create them manually. If you need to verify that they were created, run the following commands:

```
oc get configmap acm-placement -n openshift-gitops -o yaml
oc get role openshift-gitops-applicationset-controller-placement -n openshift-gitops
oc get rolebinding openshift-gitops-applicationset-controller-placement -n openshift-
gitops
oc get policy <gitopscluster-name>-argocd-policy -n openshift-gitops
```

1. Verify that the ArgoCD Policy was created on the hub cluster by running the following command:

```
oc get policy gitops-clusters-argocd-policy -n openshift-gitops
```

Customizing the Argo CD policy for the ManifestWork -based pull model

Customize the auto-generated ArgoCD Policy to add the RBAC that managed clusters need for the

ManifestWork-based pull model.

Required access: Cluster administrator

The auto-generated ArgoCD Policy configures the ArgoCD custom resource on managed clusters. This Policy is for spoke cluster configuration only, such as RBAC and ArgoCD settings.

By default, the Red Hat OpenShift GitOps add-on might not provide sufficient privileges for the OpenShift GitOps controllers on managed clusters, especially when managing resources outside the openshift-gitops namespace. Export the auto-generated Policy, add a ClusterRoleBinding, and re-apply it. The following example shows a complete Policy with RBAC:

```
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: gitops-clusters-argocd-policy
  namespace: openshift-gitops
  annotations:
    policy.open-cluster-management.io/standards: NIST-CSF
    policy.open-cluster-management.io/categories: PR.PT Protective Technology
    policy.open-cluster-management.io/controls: PR.PT-3 Least Functionality
spec:
  remediationAction: enforce
  disabled: false
  policy-templates:
    - objectDefinition:
        apiVersion: policy.open-cluster-management.io/v1
        kind: ConfigurationPolicy
        metadata:
          name: gitops-clusters-argocd-config-policy
        spec:
          pruneObjectBehavior: None
          remediationAction: enforce
          severity: medium
          object-templates:
            - complianceType: musthave
              objectDefinition:
                apiVersion: argoproj.io/v1beta1
                kind: ArgoCD
                metadata:
                  name: acm-openshift-gitops
                  namespace: openshift-gitops
                  labels:
                    apps.open-cluster-management.io/gitopsaddon: "true"
                spec:
                  server:
                    enabled: false
    - objectDefinition:
        apiVersion: policy.open-cluster-management.io/v1
        kind: ConfigurationPolicy
```

```
        metadata:
          name: gitops-clusters-argocd-rbac-policy
        spec:
          pruneObjectBehavior: None
          remediationAction: enforce
          severity: medium
          object-templates:
            - complianceType: musthave
              objectDefinition:
                apiVersion: rbac.authorization.k8s.io/v1
                kind: ClusterRoleBinding
                metadata:
                  name: acm-openshift-gitops-cluster-admin
                roleRef:
                  apiGroup: rbac.authorization.k8s.io
                  kind: ClusterRole
                  name: cluster-admin
                subjects:
                  - kind: ServiceAccount
                    name: acm-openshift-gitops-argocd-application-controller
                    namespace: openshift-gitops
```

Apply the YAML sample by running the following command:

```
oc apply -f policy.yaml
```

Important: The `cluster-admin ClusterRole` in this example grants full cluster administrator privileges. This is only an example. For production environments, see the Red Hat OpenShift GitOps documentation for configuring the Argo CD application controller with appropriate least-privilege permissions for managing resources outside its namespace.

Deploying an application with the `ManifestWork`-based pull model

Deploy applications across managed clusters by using an `ApplicationSet` resource that targets the `ManifestWork`-based pull model.

Required access: Cluster administrator

In the `ManifestWork`-based pull model, applications are deployed using an `ApplicationSet` resource on the hub cluster with specific pull-model annotations and labels. The propagation controller detects these annotations and creates `ManifestWork` resources to distribute the `Application` to managed clusters, where the local Argo CD instance reconciles it.

Complete the following steps:

1. Create a separate `Placement` resource for the `ApplicationSet` to select which managed clusters receive the application. This `Placement` is distinct from the `GitOpsCluster Placement` created

earlier. See Configuring Placement resources for GitOps for details. Add the following YAML sample:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: app-placement
  namespace: openshift-gitops
spec:
  tolerations:
  - key: cluster.open-cluster-management.io/unreachable
    operator: Exists
  - key: cluster.open-cluster-management.io/unavailable
    operator: Exists
  predicates:
  - requiredClusterSelector:
      labelSelector:
        matchExpressions:
        - key: vendor
          operator: "In"
          values:
          - OpenShift
        - key: local-cluster
          operator: DoesNotExist
```

2. Apply the YAML sample by running the following command:

```
oc apply -f app-placement.yaml
```

3. Create the `ApplicationSet` resource with the pull-model annotations and labels. The following annotations and labels are required for the `ManifestWork`-based pull model:
   - `apps.open-cluster-management.io/pull-to-ocm-managed-cluster: "true"` (label) - enables the propagation controller to distribute the `Application` via `ManifestWork`.
   - `apps.open-cluster-management.io/ocm-managed-cluster: '\{{name}\}'` (annotation) - specifies which managed cluster receives the `Application`.
   - `argocd.argoproj.io/skip-reconcile: "true"` (annotation) - prevents the hub Argo CD from reconciling the `Application` (only the spoke Argo CD reconciles it).
   - `apps.open-cluster-management.io/ocm-managed-cluster-app-namespace` (annotation) - specifies the namespace on the managed cluster where the `Application` object is created.
   - `destination.server: https://kubernetes.default.svc` - the `Application` is reconciled locally by the managed cluster Argo CD.

See the following YAML sample:

```
apiVersion: argoproj.io/v1alpha1
```

```
kind: ApplicationSet
metadata:
  name: guestbook-appset
  namespace: openshift-gitops
spec:
  generators:
  - clusterDecisionResource:
      configMapRef: acm-placement
      labelSelector:
        matchLabels:
          cluster.open-cluster-management.io/placement: app-placement
      requeueAfterSeconds: 30
  template:
    metadata:
      name: '{{name}}-guestbook'
      labels:
        apps.open-cluster-management.io/pull-to-ocm-managed-cluster: "true"
      annotations:
        argocd.argoproj.io/skip-reconcile: "true"
        apps.open-cluster-management.io/ocm-managed-cluster: '{{name}}'
        apps.open-cluster-management.io/ocm-managed-cluster-app-namespace: openshift-
gitops
    spec:
      project: default
      source:
        repoURL: https://github.com/argoproj/argocd-example-apps
        targetRevision: HEAD
        path: guestbook
      destination:
        server: https://kubernetes.default.svc
        namespace: guestbook
      syncPolicy:
        automated:
          prune: true
          selfHeal: true
        syncOptions:
        - CreateNamespace=true
```

Note: The `destination.server: https://kubernetes.default.svc` and the pull-model annotations are what distinguish the `ManifestWork`-based pull model from the other models. The propagation controller strips the annotations, rewrites the destination to in-cluster, and wraps the `Application` in a `ManifestWork` for delivery to the managed cluster. The `configMapRef: acm-placement` references the duck-type `ConfigMap` that the `GitOpsCluster` controller creates automatically during reconciliation.

1. Apply the YAML sample by running the following command:

```
oc apply -f applicationset.yaml
```

2. Verify the deployment by running the following command on the hub:

```
oc get applications.argoproj.io -n openshift-gitops
```

Additional resources

- Continue setting up the Red Hat OpenShift GitOps add-on by completing *Managing the Red Hat OpenShift GitOps add-on*.

Managing the Red Hat OpenShift GitOps add-on

Manage the lifecycle of the Red Hat OpenShift GitOps add-on on your managed clusters, including installation options, skip annotations, verification, and uninstall.

The Red Hat OpenShift GitOps add-on automates the deployment and lifecycle management of Red Hat OpenShift GitOps on the managed clusters.

The OpenShift GitOps add-on supports the following configuration options in the `gitopsAddon` specification:

`enabled`
: Enable or disable the GitOps add-on. The default is `false`.

`gitOpsOperatorImage`
: The only custom container image for the GitOps operator that you need to specify.

`overrideExistingConfigs`
: Override existing `AddOnDeploymentConfig` values with new values from the `GitOpsCluster` specification. If you use the default value of `false`, existing config values are preserved and only new values are added. If you set the value to `true`, config values from the `GitOpsCluster` specification override existing values.

`olmSubscription`
: The Operator Lifecycle Manager subscription configuration. If you set `olmSubscription.enabled` to `true`, the add-on agent uses the Operator Lifecycle Manager subscription mode for the operator installation, regardless of whether the agent automatically detects an OpenShift Container Platform cluster or not.

`argoCDAgent`
: Argo CD Agent configuration (`enabled`, `serverAddress`, `serverPort`, `mode`). The default mode is `managed`.

See the following topics to manage the add-on lifecycle:

- Enabling a deployment based on Operator Lifecycle Manager
- Skipping the OpenShift GitOps add-on enforcement
- Skipping the Argo CD policy recreation

- Uninstalling the OpenShift GitOps add-on
- Verifying the OpenShift GitOps add-on functions
- Verifying the Argo CD Agent function

Enabling a deployment based on Operator Lifecycle Manager

For OpenShift Container Platform managed clusters, enable Operator Lifecycle Manager-based deployment when add-on auto-detection of the cluster type is not working properly.

Required access: Cluster administrator

For OpenShift Container Platform managed clusters, you can enable OLM-based deployment by adding `olmSubscription.enabled: true` to the `gitopsAddon` specification in case the add-on auto-detection of the cluster type is not working properly:

```
gitopsAddon:
  enabled: true
  olmSubscription:
    enabled: true
```

Skipping the OpenShift GitOps add-on enforcement

Skip enforcement for specific resources by adding the `gitops-addon.open-cluster-management.io/skip` annotation so that the add-on does not update or manage those resources.

Required access: Cluster administrator

You can skip enforcement for specific resources by adding the `gitops-addon.open-cluster-management.io/skip` annotation to resources on the managed cluster. When a resource has this annotation, the add-on does not update or manage it.

```
apiVersion: argoproj.io/v1beta1
kind: ArgoCD
metadata:
  name: openshift-gitops
  namespace: openshift-gitops
  annotations:
    gitops-addon.open-cluster-management.io/skip: "true"
```

Skipping the Argo CD policy recreation

Prevent the controller from recreating a deleted `ArgoCD Policy` by setting an annotation on the `GitOpsCluster` resource.

Required access: Cluster administrator

To prevent the controller from recreating a deleted `ArgoCD Policy`, set the `apps.open-cluster-management.io/skip-argocd-policy: "true"` annotation on the `GitOpsCluster`. When set, the `ArgoCDPolicyReady` condition is set to `True` with `Reason=Skipped`.

Uninstalling the OpenShift GitOps add-on

Uninstall the Red Hat OpenShift GitOps add-on in the correct order so that dependencies and finalizers are cleaned up successfully.

Required access: Cluster administrator

To completely uninstall the Red Hat OpenShift GitOps add-on, complete the following steps in order. The order matters because resources have dependencies and finalizers that require proper cleanup sequencing.

Important: Plan for the uninstall to take several minutes. The add-on framework runs pre-delete cleanup Jobs on each managed cluster to remove `ArgoCD` CRs, the `GitOpsService` CR, OLM Subscription/CSV, and operator resources. Do not force-remove finalizers before the cleanup completes.

1. Delete any `ApplicationSet` resources and wait for the generated `Application` resources to be removed by running the following commands:

```
oc delete applicationset <applicationset-name> -n openshift-gitops --ignore-not
-found
```

Verify all generated `Application` resources are gone:

```
oc get applications.argoproj.io -n openshift-gitops
```

1. Delete the `Placement` resources by running the following commands:

```
oc delete placement <gitops-placement-name> -n openshift-gitops --ignore-not-found
oc delete placement <app-placement-name> -n openshift-gitops --ignore-not-found
```

2. Delete the `GitOpsCluster` resource by running the following command:

```
oc delete gitopscluster <name> -n openshift-gitops --ignore-not-found
```

3. Delete the auto-generated `Policy` and `PlacementBinding` resources by running the following commands:

```
oc delete policy <name>-argocd-policy -n openshift-gitops --ignore-not-found
```

```
oc delete placementbinding <name>-argocd-policy-binding -n openshift-gitops
--ignore-not-found
```

Important: If the `Policy` is not deleted, it continues to enforce the `ArgoCD` custom resource on managed clusters even after the add-on is removed. During cleanup, the `GitOpsCluster` controller may also recreate the `Policy` before the `GitOpsCluster` is fully removed - if this happens, delete the `Policy` and `PlacementBinding` again after the `GitOpsCluster` deletion is complete:

```
oc delete policy <name>-argocd-policy -n openshift-gitops --ignore-not-found
oc delete placementbinding <name>-argocd-policy-binding -n openshift-gitops --ignore
-not-found
```

1. Wait for the `ArgoCD` custom resource to be removed from each managed cluster. The add-on cleanup Job handles this. Monitor the removal:

   ```
   oc get argocd acm-openshift-gitops -n openshift-gitops --context <managed-cluster-
   context>
   ```

2. For OpenShift Container Platform managed clusters with OLM-based deployment, verify that the OLM `Subscription` created by the add-on is also removed:

   ```
   oc get subscription.operators.coreos.com -l apps.open-cluster-
   management.io/gitopsaddon=true -A --context <managed-cluster-context>
   ```

3. If agent mode was enabled, clean up the hub-side agent resources:

   ```
   oc delete appproject --all -n <managed-cluster-name> --ignore-not-found
   oc delete secret cluster-<managed-cluster-name> -n openshift-gitops --ignore-not
   -found
   oc delete secret <managed-cluster-name>-application-manager-cluster-secret -n
   openshift-gitops --ignore-not-found
   ```

4. Clean up the `ManagedClusterSetBinding` if it is no longer needed:

   ```
   oc delete managedclustersetbinding default -n openshift-gitops --ignore-not-found
   ```

Consequences of improper uninstall:

- `Policy` **not deleted**: The `Policy` continues to enforce the `ArgoCD` CR on managed clusters, causing the operator and Argo CD to be re-created even after the add-on is removed.
- `PlacementBinding` **not deleted**: The `PlacementBinding` keeps the `Policy` bound to the `Placement`, making re-enforcement more likely.
- **Finalizers not cleared**: Residual `Application` resources with finalizers can block namespace

deletion. If needed, patch the finalizer: `oc patch application <name> -n openshift-gitops --type=json -p '[{"op":"remove","path":"/metadata/finalizers"}]'`.

- **Application workload namespaces** (for example, `guestbook`) are not cleaned up by the add-on. Delete them manually if needed.
- **`ClusterRoleBinding` leftovers**: If you added custom `ClusterRoleBinding` resources (for example, `acm-openshift-gitops-cluster-admin`) via the `Policy`, they remain on managed clusters after uninstall. Delete them manually if needed.

Verifying the OpenShift GitOps add-on functions

Verify that the `GitOpsCluster` resource and the OpenShift GitOps add-on controller are healthy.

Required access: Cluster administrator

Verifying the GitOpsCluster status

1. View the `GitOpsCluster` resource status by running the following command:

```
oc get gitopscluster <gitopscluster-name> -n openshift-gitops -o yaml
```

2. Check the status conditions for specific failure information by running the following command:

```
oc get gitopscluster <gitopscluster-name> -n openshift-gitops -o
jsonpath='{.status.conditions}' | python3 -m json.tool
```

See the following common condition types:

- `Ready`: Overall `GitOpsCluster` health.
- `PlacementResolved`: `Placement` resource status.
- `ClustersRegistered`: Cluster registration status.
- `ArgoCDPolicyReady`: `ArgoCD Policy` creation status.
- `ArgoCDAgentPrereqsReady`: Agent prerequisites, if you have enabled the agent.
- `CertificatesReady`: TLS certificate status, if you have enabled the agent.
- `ManifestWorksApplied`: `ManifestWork` propagation status, if you have enabled the agent.

Verifying the OpenShift GitOps add-on controller logs

Check the Red Hat OpenShift GitOps add-on controller logs by running the following command:

```
oc logs -n open-cluster-management-agent-addon -l app=gitops-addon --context <managed-
cluster-context>
```

Verifying the Argo CD Agent function

Verify that the Argo CD Agent principal on the hub and the agent on the managed cluster are running.

Required access: Cluster administrator

Procedure

1. On the hub, verify the agent principal is running:

```
oc get pods -n openshift-gitops -l app.kubernetes.io/name=openshift-gitops-agent-principal
```

2. On the managed cluster, verify the agent is running:

```
oc get pods -n openshift-gitops -l app.kubernetes.io/part-of=argocd-agent --context <managed-cluster-context>
```

Deploying the Argo CD *ApplicationSet* resource in any namespace (Technology Preview)

Deploy the Argo CD `ApplicationSet` resource in any namespace when you use the Pull model.

Required access: Cluster administrator

With the Argo CD pull model, you can create `ApplicationSet` resources in any namespace on your hub clusters.

Prerequisites

- Complete the procedures to deploy the Red Hat OpenShift GitOps add-on with the `ManifestWork` -based pull model. See *Deploying Argo CD with the Red Hat OpenShift GitOps add-on and* `ManifestWork`*-based pull model*.

Technology Preview: This feature is a Technology Preview feature.

See the following topics:

- Enabling the *ApplicationSet* resource in any namespace
- Deploying the *ApplicationSet* resource for standard configurations

Enabling the *ApplicationSet* resource in any namespace

Enable the Argo CD `ApplicationSet` resource so that you can create it in namespaces other than the

Argo CD installation namespace.

Required access: Cluster administrator

Complete the following steps:

1. Enable the Argo CD `ApplicationSet` resource in any namespace on the hub cluster by running the following commands:

```
git clone https://github.com/stolostron/multicloud-integrations
cd multicloud-integrations/deploy/appset-any-namespace
./setup-appset-any-namespace.sh --namespace openshift-gitops --argocd-name
openshift-gitops
```

2. Verify that the OpenShift GitOps instance restarted and is running on your hub cluster by running the following command:

```
oc get pods -n openshift-gitops
```

3. Enable the `Application` resource in any namespace on the managed clusters. Modify the auto-generated `ArgoCD Policy` from the `ManifestWork`-based pull model to add `ARGOCD_APPLICATION_NAMESPACES`, `AppProject` wildcard settings, and RBAC. The following example shows the complete modified `Policy`:

```
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: <gitopscluster-name>-argocd-policy
  namespace: openshift-gitops
  annotations:
    policy.open-cluster-management.io/standards: NIST-CSF
    policy.open-cluster-management.io/categories: PR.PT Protective Technology
    policy.open-cluster-management.io/controls: PR.PT-3 Least Functionality
spec:
  disabled: false
  remediationAction: enforce
  policy-templates:
  - objectDefinition:
      apiVersion: policy.open-cluster-management.io/v1
      kind: ConfigurationPolicy
      metadata:
        name: <gitopscluster-name>-argocd-config-policy
      spec:
        pruneObjectBehavior: None
        remediationAction: enforce
        severity: medium
        object-templates:
        - complianceType: musthave
```

```
        objectDefinition:
          apiVersion: argoproj.io/v1beta1
          kind: ArgoCD
          metadata:
            labels:
              apps.open-cluster-management.io/gitopsaddon: "true"
            name: acm-openshift-gitops
            namespace: openshift-gitops
          spec:
            controller:
              enabled: true
              env:
              - name: ARGOCD_APPLICATION_NAMESPACES
                value: '*'
            server:
              enabled: false
  - objectDefinition:
      apiVersion: policy.open-cluster-management.io/v1
      kind: ConfigurationPolicy
      metadata:
        name: <gitopscluster-name>-argocd-appset-rbac-policy
      spec:
        pruneObjectBehavior: None
        remediationAction: enforce
        severity: medium
        object-templates:
        - complianceType: musthave
          objectDefinition:
            apiVersion: argoproj.io/v1alpha1
            kind: AppProject
            metadata:
              name: default
              namespace: openshift-gitops
            spec:
              clusterResourceWhitelist:
              - group: '*'
                kind: '*'
              destinations:
              - namespace: '*'
                server: '*'
              sourceRepos:
              - '*'
              sourceNamespaces:
              - '*'
        - complianceType: musthave
          objectDefinition:
            apiVersion: rbac.authorization.k8s.io/v1
            kind: ClusterRoleBinding
            metadata:
              name: argo-admin
            subjects:
```

```
    - kind: ServiceAccount
      name: acm-openshift-gitops-argocd-application-controller
      namespace: openshift-gitops
  roleRef:
    apiGroup: rbac.authorization.k8s.io
    kind: ClusterRole
    name: cluster-admin
```

Important: The `cluster-admin ClusterRole` is only an example. See the Red Hat OpenShift GitOps documentation for appropriate least-privilege permissions.

1. Apply the YAML file by running the following command:

```
oc apply -f policy.yaml
```

Deploying the *ApplicationSet* resource for standard configurations

Deploy an `ApplicationSet` resource for standard Pull-model configurations after you enable `ApplicationSet` in any namespace.

Required access: Cluster administrator

For simple RBAC management, create the `ApplicationSet` resource in a custom namespace with the default `AppProject`:

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: helloworld
  namespace: appset-2
spec:
  generators:
  - clusterDecisionResource:
      configMapRef: acm-placement
      labelSelector:
        matchLabels:
          cluster.open-cluster-management.io/placement: app-placement
      requeueAfterSeconds: 30
  template:
    metadata:
      annotations:
        apps.open-cluster-management.io/ocm-managed-cluster: '{{name}}'
        argocd.argoproj.io/skip-reconcile: "true"
      labels:
        apps.open-cluster-management.io/pull-to-ocm-managed-cluster: "true"
      name: '{{name}}-helloworld'
```

```
    spec:
      destination:
        namespace: helloworld
        server: https://kubernetes.default.svc
      project: default
      source:
        path: helloworld
        repoURL: https://github.com/stolostron/application-samples.git
        targetRevision: HEAD
      syncPolicy:
        automated: {}
```

Additional resources

- ApplicationSet in any namespace
- Applications in any namespace

Implementing progressive rollout strategy by using *ApplicationSet* resource (Technology Preview)

Use a progressive rollout strategy with an `ApplicationSet` resource to control how applications are updated across managed clusters.

Required access: Cluster administrator

By implementing the progressive rollout strategy to your application in your cluster fleet, you have a Red Hat OpenShift GitOps-based process that makes changes safely across your entire cluster fleet. The `ApplicationSet` controller organizes the clusters into groups and processes one group at a time through the Argo CD health cycle.

Enabling the progressive rollout strategy

1. Enable progressive syncs by running the following command:

```
oc -n openshift-gitops patch argocd openshift-gitops --type='merge' -p
'{"spec":{"applicationSet":{"extraCommandArgs":["--enable-progressive-syncs"]}}}'
```

2. Restart the controller by running the following command:

```
oc -n openshift-gitops rollout restart deployment openshift-gitops-applicationset-
controller
```

Understanding a sample ApplicationSet resource for the progressive rollout strategy

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
```

```
  name: guestbook-allclusters-app-set
  namespace: openshift-gitops
spec:
  generators:
  - clusterDecisionResource:
      configMapRef: acm-placement
      labelSelector:
        matchLabels:
          cluster.open-cluster-management.io/placement: app-placement
      requeueAfterSeconds: 30
  strategy:
    type: RollingSync
    rollingSync:
      steps:
      - name: dev-stage
        matchExpressions:
        - key: envLabel
          operator: In
          values: [env-dev]
        maxUpdate: 100%
      - name: qa-stage
        matchExpressions:
        - key: envLabel
          operator: In
          values: [env-qa]
        maxUpdate: 100%
      - name: prod-stage
        matchExpressions:
        - key: envLabel
          operator: In
          values: [env-prod]
        maxUpdate: 100%
  template:
    preserveFields:
    - metadata.labels.envLabel
    metadata:
      annotations:
        apps.open-cluster-management.io/ocm-managed-cluster: '{{name}}'
        apps.open-cluster-management.io/ocm-managed-cluster-app-namespace: openshift-
gitops
        argocd.argoproj.io/skip-reconcile: "true"
      labels:
        apps.open-cluster-management.io/pull-to-ocm-managed-cluster: "true"
      name: '{{name}}-guestbook-app'
    spec:
      destination:
        namespace: guestbook
        server: https://kubernetes.default.svc
      project: default
      sources:
      - repoURL: https://github.com/argoproj/argocd-example-apps.git
```

```
        targetRevision: main
        path: guestbook
      syncPolicy:
        syncOptions:
        - CreateNamespace=true
```

Use the `preserveFields` parameter to list the labels that need to be ignored by the `ApplicationSet` controller. Without this field, the controller overwrites or deletes labels not defined in the template.

Understanding labels for the progressive rollout strategy

Ensure each application has a unique label specifying its promotion order:

```
oc -n openshift-gitops label applications.argoproj.io/cluster1-guestbook-app
envLabel=env-dev
oc -n openshift-gitops label applications.argoproj.io/cluster2-guestbook-app
envLabel=env-qa
oc -n openshift-gitops label applications.argoproj.io/cluster3-guestbook-app
envLabel=env-prod
```

Understanding changes to the Git repository

When Argo CD detects committed changes, the `ApplicationSet` controller detects `OutOfSync` applications and schedules a staged rollout. The controller processes `env-dev` first, then `env-qa`, then `env-prod` - each group must reach `Healthy` before the next begins.

Managing policy definitions with Red Hat OpenShift GitOps

Manage Red Hat Advanced Cluster Management policy definitions by using Red Hat OpenShift GitOps and Argo CD.

Required access: Cluster administrator

With the `ArgoCD` resource, you can use Red Hat OpenShift GitOps to manage policy definitions by granting OpenShift GitOps access to create policies on the hub cluster.

Prerequisites

You must log in to your hub cluster.

Creating a ClusterRole resource for OpenShift GitOps

Create a `ClusterRole` with access to create, read, update, and delete policies and placements:

```
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: openshift-gitops-policy-admin
rules:
```

```
  - verbs: [get, list, watch, create, update, patch, delete]
    apiGroups: [policy.open-cluster-management.io]
    resources: [policies, configurationpolicies, certificatepolicies,
operatorpolicies, policysets, placementbindings]
  - verbs: [get, list, watch, create, update, patch, delete]
    apiGroups: [cluster.open-cluster-management.io]
    resources: [placements, placements/status, placementdecisions,
placementdecisions/status]
```

Create a `ClusterRoleBinding`:

```
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: openshift-gitops-policy-admin
subjects:
  - kind: ServiceAccount
    name: openshift-gitops-argocd-application-controller
    namespace: openshift-gitops
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: openshift-gitops-policy-admin
```

Notes:

- Replicated policies get the `argocd.argoproj.io/compare-options: IgnoreExtraneous` annotation automatically.
- To hide replicated policies from Argo CD, set `spec.copyPolicyMetadata` to `false` on the policy definition.

Integrating the Policy Generator with OpenShift GitOps

Configure the Init Container to copy the Policy Generator binary. Your `ArgoCD` resource might resemble:

```
apiVersion: argoproj.io/v1beta1
kind: ArgoCD
metadata:
  name: openshift-gitops
  namespace: openshift-gitops
spec:
  kustomizeBuildOptions: --enable-alpha-plugins
  repo:
    env:
    - name: KUSTOMIZE_PLUGIN_HOME
      value: /etc/kustomize/plugin
    initContainers:
    - args: ["-c", "cp /policy-generator/PolicyGenerator-not-fips-compliant /policy-
```

```
generator-tmp/PolicyGenerator"]
      command: ["/bin/bash"]
      image: registry.redhat.io/rhacm2/multicluster-operators-subscription-
rhel9:v<version>
      name: policy-generator-install
      volumeMounts:
      - mountPath: /policy-generator-tmp
        name: policy-generator
    volumeMounts:
    - mountPath: /etc/kustomize/plugin/policy.open-cluster-
management.io/v1/policygenerator
      name: policy-generator
    volumes:
    - emptyDir: {}
      name: policy-generator
```

Replace `<version>` with the latest Red Hat Advanced Cluster Management version.

Configuring policy health checks in OpenShift GitOps

1. Download and apply the health checks:

```
wget https://raw.githubusercontent.com/open-cluster-management-io/policy-
collection/main/stable/CM-Configuration-Management/argocd-policy-healthchecks.yaml
```

Apply via **Governance** > **Create policy** in the console.